

Implementation Blueprint

Five-Stage Build Plan for VDR-LLM-Prolog

Registry: [\[HOWL-VDR-11-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → [\[HOWL-VDR-3-2026\]](#) → [\[HOWL-VDR-4-2026\]](#) → [\[HOWL-LLM-1-2026\]](#) → [\[HOWL-VDR-5-2026\]](#) → [\[HOWL-VDR-6-2026\]](#) → [\[HOWL-VDR-7-2026\]](#) → [\[HOWL-VDR-8-2026\]](#) → [\[HOWL-VDR-9-2026\]](#) → [\[HOWL-VDR-10-2026\]](#) → [\[HOWL-VDR-11-2026\]](#)

DOI: 10.5281/zenodo.20226180

Date: May 2026

Domain: Applied Philosophy / Systems Engineering / Implementation Planning

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Supplementary Materials:

- `supplementary/function_iose_5_stages_spec.md` — Complete function-level IOSE specification for all 65 modules across 5 stages, with inputs, outputs, side effects, and properties declared for every function.
- `supplementary/iose_spec_depth.md` — Full data structure definitions (dataclasses, enums), module map, file layout, line count estimates, Zig port strategy, and stage deliverable tables.

Abstract

Ten papers specified the VDR-LLM-Prolog system. VDR-10 provided the engineering foundation — the IOSE system model, operational principles, and comprehensive numeric builtins. This paper specifies how to build it.

The system is built in five stages, each producing a complete, testable, runnable system that handles a full lifecycle at its level of capability. Stage 1 is a toy that can create knowledge bases, assert and query facts, run Prolog rules, perform exact arithmetic, and demonstrate one training-evaluation cycle. Stage 2 adds command tokens, path addressing, scope resolution, constraints, and the scratchpad. Stage 3 adds session management, inference notebooks, Q-basis transcendentals, functional remainders, and domain-specific mathematics. Stage 4 adds operational environments, grants, filesystem and network operations, all four inference modes, and the lifecycle pipeline. Stage 5 completes the system with Docker and SSH environments, compilation, linting, feedback collection, deployment, monitoring, canary deployment, and retirement.

The build uses Python 3.8 as the prototype language, leveraging the existing VDR-1 through VDR-4 codebase (~5,000 lines of tested exact arithmetic, linear algebra, and ML stack code). New code is approximately 15,500 lines across 65 modules. Every function has an IOSE declaration — inputs, outputs, side effects, and properties — which simultaneously serves as the test specification, the documentation, and the interface contract for the eventual Zig 0.15.1 production port.

The central claim is that a system specified by ten papers, governed by operational engineering principles, and built in disciplined stages with IOSE declarations at every function is not a research prototype — it is an engineering project with a concrete, executable build plan.

1. Context: From Specification to Construction

1.1 What Has Been Specified

The VDR-LLM-Prolog system has been specified across ten papers:

Exact arithmetic (VDR-1 through VDR-4). Every number is an integer triple [V, D, R] — value, denominator, remainder. 705 tests across 23 mathematical domains, zero VDR computation errors. A working transformer with exact softmax, exact autodiff, and exact training.

Knowledge architecture (VDR-5). Everything stored in Knowledge Bases — facts, rules, constraints organized in a scoped tree. User accounts as KBs. Organizational hierarchy as the tree structure.

Execution layer (VDR-6). 255 deterministic primitives invoked through compact command tokens. Sandboxed operational environments. Positive credential grants.

Lifecycle (VDR-7). The complete model lifecycle — data sourcing through retirement — as KB operations.

Runtime state (VDR-8). Data primitives (LRU caches, counters, locks, queues, stacks, ring buffers, bitsets) as fields on the KB struct. Universal dotted-path addressing with integer ID acceleration. Session snapshots and disposable cloning.

Orchestrated Inference (VDR-9). The pattern by which the LLM composes tools into multi-step investigations with four inference modes, notebooks, provenance, and confidence propagation.

Engineering foundation (VDR-10). The IOSE system model for all components. Fifteen operational engineering principles as enforceable Prolog rules. 448 comprehensive builtins with full VDR-1 through VDR-3 mathematical capability.

1.2 What This Paper Provides

A build plan. Not a wish list — a concrete, staged, module-by-module, function-by-function plan for constructing the system. Every module has a stage assignment. Every

function has an IOSE declaration. Every stage has a test count target and a defined capability level.

The complete function-level specification lives in the supplementary materials. This paper describes the architecture, the staging strategy, the key design decisions, and the integration points. It does not reproduce the supplementary specifications — it explains them.

2. Build Philosophy

2.1 Comprehensive Then Incremental

The specification is comprehensive — ten papers define the whole system top-down. The build is incremental — five stages, each producing a testable system. At no stage is the system aggregated in the OSO sense (C18). Each stage is a complete, internally consistent system that handles a full lifecycle at its level of capability. Stage 1 handles a toy lifecycle. Stage 5 handles production. But even Stage 1 has KB creation, fact assertion, rule evaluation, exact arithmetic, and a training-evaluation loop.

2.2 Python First, Zig Final

Python 3.8 is the prototype language. It has the existing VDR-1 through VDR-4 codebase — `vdr.py`, `active_mul.py`, `fn.py`, `linalg.py`, and nineteen other modules totaling approximately 5,000 lines of tested, working code. The prototype validates every design decision in a language where iteration is fast and debugging is straightforward.

The production implementation will be Zig 0.15.1. The port is mechanical because the IOSE declarations define the interfaces — same inputs, same outputs, same side effects, same properties. Python dataclasses map to Zig structs. Python dicts map to Zig HashMaps. Python lists map to Zig ArrayLists. Python enums map to Zig enums. The prototype is designed for portability from the start.

The Zig port happens after Stage 5 validates the complete system in Python. The IOSE declarations serve simultaneously as: the Python function signatures, the test specifications, the documentation, and the Zig interface contracts.

2.3 IOSE At Every Function

Every function in the system — from `vdr_add` to `create_deployment` to `session_clone` — has an IOSE declaration before it has an implementation. The declaration states what goes in, what comes out, and what else changes. This discipline starts at Stage 1 and never relaxes.

The IOSE declaration is the test specification. To test a function: provide the declared inputs, verify the declared outputs, confirm the declared side effects occurred, and verify the declared properties hold (determinism by running twice and comparing, idempotency by running the output through the function again, commutativity by swapping arguments).

2.4 Existing Code Integration

The VDR-1 through VDR-4 codebase is not rewritten. It is wrapped. The existing VDR class in `vdr.py` becomes the implementation behind the `vdr_add`, `vdr_mul`, `vdr_div` builtins. The existing `Vec` and `Mat` classes in `linalg.py` become the implementation behind the linear algebra builtins. The existing `softmax` function becomes the implementation behind `vdr_softmax`. The wrapping layer adds IOSE declarations, type dispatch, and error handling via the `Result` type. The core mathematics is proven — 705 tests — and is not touched.

3. Module Architecture

3.1 Ten Layers

The system organizes into ten layers, each a directory containing related modules:

core/ — The existing VDR arithmetic library plus new type dispatch and error handling. This is the mathematical foundation. Five existing modules (`vdr.py`, `active_mul.py`, `fn.py`, `linalg.py`, `export.py`) plus two new modules (`types.py`, `errors.py`).

kb/ — The Knowledge Base engine. KB struct, fact storage, Prolog-style rule engine with unification and backtracking, constraint checking, scope resolution, working data bindings. Six modules.

path/ — The universal addressing system. Path-to-integer registry, dotted path resolution with relative paths, mount system with cycle detection. Three modules.

primitives/ — All 448 builtins as IOSE-declared functions. Organized by category: text, collections, arithmetic, comparison, rounding, number theory, linear algebra, statistics, probability, conversion, time, identity, graphs, logic, integer operations, active arithmetic, structure operations, Q-basis, functional remainder, discrete calculus, denominator management, polynomial, finite field, Markov, graph math. Twenty-eight modules.

data_primitives/ — Runtime state data structures. Counter, lock, queue, stack, LRU cache, ring buffer, bitset. Each module provides the IOSE-declared functions that operate on the corresponding data structure within a KB. Seven modules.

command/ — Command token parsing and execution. Token type definitions, stream parser, dispatch executor, scratchpad. Four modules.

session/ — Session state management. Snapshot capture and restore, clone creation and destruction, lifecycle operations (`reset`, `diff`, `info`). Three modules.

inference/ — Orchestrated Inference. Notebook creation and templating, the assess-formalize-execute-store loop, the four inference modes (deductive, inductive, abductive, analogical), confidence propagation, provenance tracking and challenge mechanism. Five modules.

env/ — Operational environments. Abstract base interface, local execution, Docker container management, SSH remote execution, VM management. Five modules.

ops/ — Operational primitives. Filesystem operations, script execution, compilation, linting, network operations, process management, grant verification. Seven modules.

lifecycle/ — Model lifecycle management. Data pipeline (source registry, corpus preparation, tokenization), training orchestration, evaluation, feedback collection, deployment, monitoring. Six modules.

iose/ — IOSE infrastructure. Declaration registry, chain validation, and OSO principle loading. Three modules.

Total: 65 modules across 12 directories (including tests/).

3.2 Dependency Flow

The layers have a strict dependency order. Lower layers never import from higher layers.

```
core (no dependencies)
    ↓
kb (depends on core)
    ↓
path (depends on kb)
    ↓
data_primitives (depends on kb)
    ↓
primitives (depends on core, kb)
    ↓
command (depends on primitives, path, kb)
    ↓
iose (depends on kb, primitives)
    ↓
session (depends on kb, data_primitives, path)
    ↓
```

inference (depends on kb, primitives, command, session)

↓

env (depends on kb, command)

↓

ops (depends on env, kb, command)

↓

lifecycle (depends on ops, kb, primitives, inference)

This ordering means Stage 1 (core + kb + primitives + data_primitives + iose) has no upward dependencies. Each subsequent stage adds layers that depend only on layers from the current or earlier stages. No circular dependencies. No forward references.

4. The Five Stages

4.1 Stage 1: Toy Full Lifecycle

Capability: Create KBs, assert and query facts, run Prolog rules with unification and backtracking, perform exact VDR arithmetic, use counters/locks/queues/stacks, and demonstrate one complete lifecycle pass: initialize model → train one step → checkpoint → evaluate → report. Everything works. Nothing is fast. Nothing scales. The full loop exists.

Modules: 24 total.

The core layer copies in the existing VDR-4 codebase (vdr.py, active_mul.py, fn.py, linalg.py) and adds types.py for the number type hierarchy and dispatch, and errors.py for the Result wrapper.

The kb layer introduces the KB struct with all 25 fields from VDR-10, the fact store with idempotent assertion and retraction, the Prolog rule engine with unification over atoms, variables, VDR fractions, and lists, and working data bindings with parent-chain inheritance.

The primitives layer introduces the first wave of builtins: closed arithmetic (8 builtins wrapping vdr.py), comparison (10), rounding and extraction (7), list aggregates (8), text operations (17), collection operations (36), set operations (14), mapping operations (15), conversion operations (14 including the critical conversion boundaries), logic operations (11), and integer fast path with bit operations (21). Approximately 150 builtins active.

The data_primitives layer introduces counter, lock, queue, and stack — the four simplest runtime state structures. Each has a module with IOSE-declared functions for create,

read, write, and query operations.

The iose layer introduces the builtin registry (register, lookup by ID and name, category listing) and the OSO principles KB loader (15 axioms, core knowability facts, priority rules).

The toy lifecycle:

```
# 1. Create KB tree

root = create_kb("root", "root")

model = create_kb("model", "root.model", parent_id=root.id)

train = create_kb("train", "root.train", parent_id=root.id)

# 2. Assert architecture

assert_fact(model, Fact("hidden_dim", [2]))

assert_fact(model, Fact("vocab_size", [3]))

# 3. Initialize weights (wrapping VDR-4 init.py)

weights = initialize_model(model, seed=42)

# 4. Train one step (wrapping VDR-4 trainer.py)

train.counters["step"] = Counter(min_value=0)

loss = train_step(model, train, batch, lr=VDRFraction(1, 1000, 0))

train.counters["step"].inc()

# 5. Checkpoint

checkpoint = create_checkpoint(model, train, step=1)

# 6. Evaluate

accuracy = evaluate(model, test_data)

eval_kb = create_kb("eval", "root.eval", parent_id=root.id)

assert_fact(eval_kb, Fact("accuracy", [accuracy]))
```

7. Query result

```
result = query_facts(eval_kb, "accuracy", [None])
```

Every value is an exact VDR fraction. Every fact is in a KB. Every step is logged. The lifecycle is toy-sized but structurally complete.

Tests: 150 target. Core arithmetic (30), KB operations (30), data primitives (20), text/collections/sets/mappings (40), lifecycle integration (10), IOSE registry (10), OSO principles (10).

4.2 Stage 2: Upgraded Toy

Capability: Command tokens replace direct API calls. The system can parse a mixed stream of text and CMD: tokens, dispatch each command to the appropriate builtin, and return results. Path addressing gives every KB a dotted path with integer ID acceleration. Scope resolution enables topic switching — activating a topic makes its KB chain visible and deactivates others. Constraints fire on violations. The scratchpad provides an internal computation channel.

New modules: 13 (cumulative 37).

The kb layer gains the constraint engine (evaluate Prolog conditions, enforce violation actions) and the scope resolver (build scope chains by walking parent IDs, scoped query with first-match-wins semantics, scoped binding resolution with inheritance).

The path layer introduces the path registry (assign integer IDs, resolve dotted paths) and the path resolver (parent, children, ancestors, depth, common ancestor).

The command layer introduces command token types, the stream parser (split text from CMD: lines), the executor (resolve paths to IDs, validate types against IOSE declarations, dispatch to builtins), and the scratchpad (a RingBuffer-based internal workspace).

The primitives layer gains the second wave: active arithmetic (5), structure operations (3 — lift, rebase, projection), number theory (13), linear algebra (24 wrapping linalg.py), statistics (16), probability (included in statistics), time operations (10), identity operations (8), and graph algorithms (13). Total builtins reaches approximately 300.

The data_primitives layer gains LRU cache, ring buffer, and bitset — the three remaining runtime state structures.

The iose layer gains the validator (type compatibility checking across chains, side effect preview, post-execution contract verification).

What changes architecturally:

The system moves from Python API calls to structured command tokens. The LLM (or test harness) emits `CMD: vdr_add(root.train.loss_step_0, 1/10)` and the executor resolves the path, looks up the data, invokes the builtin, and returns the result. The scratchpad holds intermediate computations. The scope resolver ensures that switching from the training topic to the evaluation topic changes which facts are visible.

Constraints become active. A constraint like `loss_finite` with condition `loss < 1000` is checked after each training step. If violated, the `on_violation` action fires — which might halt training, log a warning, or escalate.

Tests: +200 (cumulative 350). Command token parsing and execution (40), path registry and resolution (30), scope resolver (30), constraint engine (20), new builtins (60), new data primitives (20).

4.3 Stage 3: Capacity Building

Capability: Session snapshots capture all live state atomically. Clones fork independent sessions from snapshots. Inference notebooks house structured investigations with the assess-formalize-execute-store loop. Q-basis operations manipulate transcendental constants. Functional remainders compute `sqrt`, `exp`, `log`, `sin`, `cos` at arbitrary depth. Discrete calculus provides exact derivatives and integrals. Domain-specific math covers polynomials, finite fields, Markov chains, and graph mathematics. Denominator management tracks and controls growth. The mount system allows cross-branch KB references.

New modules: 12 (cumulative 49).

The path layer gains the mount system — create mounts with cycle detection, resolve queries through mounts respecting read-only/read-write/snapshot/mirror modes.

The primitives layer gains the third wave: Q-basis operations (7), functional remainder operations (8), discrete calculus (6), denominator management (5), polynomial operations (8), finite field operations (4), Markov chain operations (3), and graph math (2). Total builtins reaches approximately 400.

The session layer introduces snapshot capture and restore, clone creation and destruction, and lifecycle operations (`reset`, `diff`, `info`, `list`).

The inference layer introduces notebook creation (with template support for common investigation types), the inference loop (assess → formalize → execute → store with budget management and stall detection), confidence propagation (deductive, inductive, abductive, analogical formulas), and provenance tracking (evidence recording, conclusion recording, derivation tracing, challenge mechanism).

What changes architecturally:

The system gains memory and recovery. Before Stage 3, a bad state required starting over. Now the system can snapshot a known-good state, experiment, and restore if things go wrong. The disposable clone pattern becomes available — snapshot a stable operator, launch workers from it, kill them when they drift, launch fresh ones.

The system gains investigation capability. An inference notebook formalizes a multi-step investigation with declared goals, resource budgets, and progress tracking. The loop cycles through assessment (what do I know? what's missing?), formalization (write Prolog rules, assemble primitive chains), execution (run the formalized step), and storage (persist results to KB). Budget constraints prevent runaway investigations. Stall detection forces backtracking or conclusion when progress stops.

The system gains the full VDR-1 through VDR-3 mathematical capability. Q-basis lets the system add $\pi + e$ with one integer addition. Functional remainders let it compute $\sqrt{2}$ to 150 digits. Discrete calculus lets it take exact derivatives and integrals. Denominator management lets it monitor and control the growth that accumulates through arithmetic operations.

Tests: +250 (cumulative 600). Session management (30), inference notebook and loop (40), confidence (20), Q-basis (20), functional remainder (30), discrete calculus (20), domain math (40), mount system (20), denominator management (15), integration (15).

4.4 Stage 4: Full Integration

Capability: The system can execute code in sandboxed environments (local first), fetch external data via network operations, manage files, run background processes, and authorize everything through positive credential grants. All four inference modes are implemented with their characteristic tool signatures. The lifecycle pipeline handles data sourcing, corpus preparation, tokenization, model initialization, training, checkpointing, and evaluation. The system is functionally complete except for remote environments and production deployment features.

New modules: 9 (cumulative 58).

The env layer introduces the abstract environment interface (10 methods that every environment implements) and the local environment (execute via Python subprocess on the host machine, for development and testing).

The ops layer introduces filesystem operations (15 builtins), script execution (5), network operations (5), process management (7), and the grant system (store, verify, use, list effective grants through KB hierarchy).

The inference layer gains the mode implementations — deductive (assert premises and rules, query Prolog for derivation), inductive (gather evidence, write scoring rules, rank hypotheses), abductive (assert observations and causal rules, query for explanations), and analogical (assert source and target domain structures, query for structural mappings).

The lifecycle layer introduces the data pipeline (source registration, corpus preparation with filters, tokenization with vocabulary freezing), training orchestration (model initialization, train step wrapping VDR-4 trainer, checkpoint creation and restoration), and evaluation (benchmark running, eval suite, cross-checkpoint comparison).

What changes architecturally:

The system gains hands. Before Stage 4, it could compute and reason but could not interact with the outside world. Now it can read and write files, execute scripts in a sandboxed environment, fetch data from HTTP endpoints, and manage background processes. Every external operation requires a grant — no operation executes without explicit authorization.

The inference system gains its full power. The four modes compose: an investigation might start with abductive inference (what could cause these symptoms?), switch to inductive (score hypotheses against evidence), then deductive (derive implications of the

leading hypothesis). The external data integration pipeline from VDR-9 — acquire, parse, convert, store, index, process — is now executable because the network and filesystem operations exist.

The lifecycle pipeline means the system can manage its own training. Initialize a model from a KB architecture specification, train it with exact gradients and exact parameter updates, checkpoint the exact state, and evaluate against benchmarks. The entire pipeline from data source to evaluation result is one KB tree.

Tests: +300 (cumulative 900). Grant system (20), filesystem (30), script execution (20), network (15), process management (15), inference modes (40), lifecycle pipeline (60), local environment (20), integration (40), IOSE contract verification (40).

4.5 Stage 5: Production Completion

Capability: The system is complete. Docker and SSH environments provide production-grade sandboxing and remote execution. Compilation primitives build code in Zig, C, and Rust. Linting primitives analyze code quality. The feedback system collects human preferences and trains reward models. The deployment system manages serving configuration, canary deployment, and rollback. The monitoring system tracks runtime metrics, detects drift, and triggers alerts. The retirement system archives model KBs for long-term audit. Every feature specified in VDR-1 through VDR-10 is implemented.

New modules: 7 (cumulative 65).

The env layer gains Docker (container creation, start, stop, destroy), SSH (remote command execution via key authentication), and VM (virtual machine management) environments. All implement the same 10-method interface — the command token executor does not change.

The ops layer gains compilation (4 builtins) and linting (8 builtins).

The lifecycle layer gains feedback collection (pairwise judgment storage, agreement computation, reward model training, DPO alignment), deployment (deployment configuration, canary creation, promotion, rollback), and monitoring (watch creation, metric recording, drift detection).

What changes architecturally:

The system gains production readiness. Docker isolation means user-generated Python scripts run in containers, not on the host. SSH means training can happen on GPU clusters. Canary deployment means model updates roll out gradually with automatic rollback on regression. Monitoring means the system tracks its own health.

The feedback loop closes. The deployed model generates responses. Humans judge them. The judgments train a reward model or feed DPO directly. The aligned model is evaluated, canary-deployed, monitored, and — when superseded — retired with its complete KB lineage preserved for audit.

Tests: +350 (cumulative 1,250). Docker (40), SSH (30), VM (20), compilation (15), linting (20), feedback (30), deployment (30), monitoring (30), canary and rollback (25),

retirement (15), full lifecycle integration (40), end-to-end (25), Zig port preparation (30).

5. Cross-Stage Invariants

These hold at every stage. They are tested at every stage. They are never relaxed.

IOSE declared. Every function has an IOSE declaration before it has an implementation. The declaration is written first. The implementation satisfies it. The test verifies it.

Exact arithmetic. Every numeric operation uses VDR fractions or exact integers. No floats anywhere in the computation path. The only place floats appear is at the declared conversion boundary when external data enters the system — and that conversion is logged with an exact error bound.

KB is truth. All persistent state lives in KBs. No module globals holding state. No hidden caches. The KB tree is the single source of truth. The OSO principle: the KB is the model for control (C39), not a model for understanding (C38).

Data primitives bounded. Every counter, queue, stack, ring buffer, LRU cache, and bitset has a declared maximum capacity. No unbounded growth. The capacity is set at creation and enforced at every mutation.

No silent truncation. Every precision reduction is declared with an exact error bound. The VDR system never silently discards information. If a denominator is too large and must be reprojected to Q-basis, the reprojection logs the exact error. If a fraction is displayed as a decimal, the display function is marked lossy and the generating fraction is always available.

Tests pass. All tests from the current stage and all prior stages pass. The test suite is cumulative. Stage 3's tests include all of Stage 1's and Stage 2's tests, unchanged and passing.

OSO principles loaded. The root.system.oso KB is loaded at startup with all 15 axioms, approximately 80 facts, approximately 60 rules, and 21 constraints. The principles are not documentation — they are active Prolog rules that the inference loop queries and the constraint system enforces.

Idempotent where declared. Every operation tagged with the idempotent property verifies $f(f(x)) = f(x)$ in its test suite. Session restore, bitset set, counter set, KB assert of existing fact, lock release on free lock — all tested for idempotency.

One canonical method. Each task category has exactly one canonical builtin. There is one way to sort a list, one way to assert a fact, one way to execute code in an environment. The builtin registry enforces uniqueness — no two builtins in the same category perform the same task.

6. The Existing Codebase

The build does not start from zero. VDR-1 through VDR-4 produced a working Python codebase of approximately 5,000 lines across 24 modules:

Arithmetic (5 modules): `vdr.py` (core triple, closed arithmetic, normalization, equality), `active_mul.py` (active multiplication and division), `fn.py` (functional remainders, discrete calculus), `linalg.py` (Vec, Mat, determinant, inverse, solve), `export.py` (lossy boundary to float/decimal).

Transcendental (3 modules): `exp.py` (exact fraction exponential), `logarithm.py` (exact fraction logarithm), `basis.py` (Q335 shared denominator).

ML stack (4 modules): `softmax.py` (exact softmax, rational surrogate), `autodiff.py` (reverse-mode exact differentiation), `nn.py` (linear layers, ReLU, sequential), `losses.py` (MSE, L1, hinge).

Infrastructure (8 modules): `optim.py` (SGD, momentum), `rng.py` (deterministic PRNG), `init.py` (rational initialization), `sampling.py` (categorical, top-k, nucleus), `datasets.py` (tokenization, batching), `metrics.py` (accuracy, denominator tracking), `checkpoint.py` (exact save/load), `tensor.py` (batched operations).

Architecture (4 modules): `attention.py` (scores, masking, weighting), `transformer.py` (embedding, blocks, LM head), `trainer.py` (training loops).

This code has 705 passing tests with zero VDR computation errors. It is not rewritten. It is wrapped in IOSE-declared builtin functions at Stage 1, then progressively exposed through the command token system at Stage 2.

7. New Code Estimates

Excluding the existing VDR-4 codebase, the new code breaks down by stage:

Stage 1: ~2,800 lines. KB struct and fact store (~350), Prolog rule engine with unification (~400), working data (~100), type hierarchy and dispatch (~200), error types (~50), primitive wrappers for arithmetic, comparison, rounding, aggregates (~400), text/collections/sets/mappings primitives (~900), conversion primitives (~200), logic primitives (~150), integer and bit operations (~150), data primitive functions (~200), IOSE registry (~150), OSO principles loader (~200).

Stage 2: ~3,200 lines. Constraint engine (~250), scope resolver (~200), path registry (~200), path resolver (~150), command token types (~80), command parser (~200), command executor (~250), scratchpad (~60), IOSE validator (~200), active arithmetic and structure ops (~220), number theory (~250), linalg builtins (~200), statistics and probability (~400), time operations (~150), identity operations (~120), graph algorithms (~300), LRU/ring/bitset data primitives (~200).

Stage 3: ~3,000 lines. Mount system (~200), Q-basis operations (~200), functional remainder operations (~250), discrete calculus (~200), denominator management (~100),

polynomial operations (~250), finite field operations (~100), Markov operations (~150), graph math (~100), session snapshot (~200), session clone (~150), session lifecycle (~150), inference notebook (~200), inference loop (~300), confidence propagation (~150), provenance (~200).

Stage 4: ~3,500 lines. Environment base interface (~200), local environment (~300), filesystem operations (~300), script execution (~200), network operations (~200), process management (~250), grant system (~200), inference modes (~400), data pipeline (~300), training orchestration (~400), evaluation (~250).

Stage 5: ~3,000 lines. Docker environment (~400), SSH environment (~300), VM environment (~200), compilation (~150), linting (~200), feedback collection (~350), deployment (~400), monitoring (~400), retirement (included in deployment).

Total new code: ~15,500 lines. Combined with the ~5,000 lines of existing VDR-4 code: approximately 20,500 lines for the complete Python prototype.

8. The Builtin Registration Pattern

Every primitive module follows the same pattern. This uniformity is deliberate — it makes bulk registration mechanical and ensures no builtin escapes without an IOSE declaration.

The pattern: define a table of (id, name, inputs, outputs, side_effects, properties, description, implementation). Call a registration helper that creates the IOSEDeclaration and BuiltinDef, then adds it to the global registry. One table per module. One registration call per module.

Categories that share identical structure — pure functions with no side effects, same input/output shape — use the same table format. The 17 text builtins, the 36 collection builtins, the 14 set builtins, the 15 mapping builtins, the 10 comparison builtins, and the 8 identity builtins all use the same registration pattern with different tables. The implementation details differ. The registration is uniform.

Categories with KB-internal side effects — data primitives (53 builtins), KB operations (15), path and mount (17), session (8) — use a variant that declares the side effects. The pattern is the same table-driven registration with an additional side_effects column.

Categories wrapping the existing VDR-4 code — closed arithmetic (8), linear algebra (24), statistics (16) — delegate to the existing Python classes. The builtin function is a thin wrapper that converts between VDRFraction dataclasses and the existing VDR class instances.

The operational categories — filesystem (15), compilation (4), execution (5), linting (8), network (5), process (7) — use a pattern that includes grant verification before dispatch. Each function first calls `grant_store.verify_grant(operation_class, operation_type, location, user)` and only proceeds if a valid grant is returned.

9. The Prolog Rule Engine

The rule engine is the most complex new module in Stage 1. It implements Prolog-style depth-first search with backtracking, supporting unification over five term types:

Atoms (strings) unify by equality. “softmax” unifies with “softmax” and nothing else.

Variables (strings starting with “?”) unify with anything and bind to the matched value. “?X” unifies with “softmax” and binds ?X = “softmax”.

VDR fractions unify by normalized value equality. [1, 2, 0] unifies with [2, 4, 0] because both equal 1/2. The comparison uses cross-multiplication of exact integers — no floats.

Lists unify element-wise. [1, ?X, 3] unifies with [1, 2, 3] binding ?X = 2. Different-length lists do not unify.

Facts (predicate + args) unify if predicates match and all arguments unify recursively.

The engine evaluates a goal against a KB’s facts and rules. For each fact matching the goal’s predicate, it attempts unification. If unification succeeds, the bindings are returned. For each rule whose head matches the goal, the engine recursively evaluates the body goals, threading bindings through. Backtracking occurs when a body goal fails — the engine undoes the most recent choice and tries the next alternative.

The engine has a depth limit (default 100) to prevent infinite recursion. It supports `findall` (collect all solutions) and first-solution-only semantics (cut after first match, used for scoped queries where the first in-scope match wins).

10. Key Design Decisions

10.1 The Result Type

Every function that can fail returns a `Result` — either `Ok(value)` or `Err(VDRError)`. No exceptions for expected failures. Division by zero, missing key, empty list, out of range — these are not exceptional. They are `Err` results with descriptive error codes.

This is an OSO principle in practice: operational logic handles failure explicitly (D4, C31). The Result type makes every failure path visible in the function signature. No hidden exception that might propagate unhandled.

10.2 The KB Store

All KBs live in a global store — a dictionary mapping integer IDs to KnowledgeBase instances. The store is the single source of truth. Every function that needs a KB takes the store as a parameter (or accesses it through a well-known path). No KBs exist outside the store.

This is the OSO “model for control” principle (C39). The KB store is the control model. The LLM’s context window is the understanding model. When they disagree, the store wins.

10.3 Turn Counter

A global turn counter increments with each user interaction. Every mutation (fact assertion, counter increment, lock acquisition) records the turn number. Every snapshot records the turn count. Every evidence record includes the turn when the evidence was acquired. The turn counter is the system clock — it provides total ordering of events without relying on wall-clock time, which may not be deterministic across platforms.

10.4 Deep Copy for Snapshots

Session snapshots deep-copy all live state. A snapshot is completely independent of the source — modifying the source after snapshot does not affect the snapshot, and restoring the snapshot does not reference the source. This is essential for the disposable clone pattern: clones must be independent. Python’s `copy.deepcopy` handles this in the prototype. The Zig port uses explicit allocation and copy.

10.5 Functional Remainder Resolution

Functional remainders (`FnRemainder`) are not automatically resolved. They sit as callable objects in the remainder slot until `fn_resolve(fn_remainder, depth)` is explicitly called. This is because resolution at different depths produces different exact rationals, and the appropriate depth depends on the application. The LLM or the calling code chooses the depth. The system never silently resolves at an arbitrary depth.

11. Integration Points Between Stages

11.1 Stage 1 → Stage 2

The primitives registered in Stage 1 become dispatchable through command tokens in Stage 2. The executor looks up builtins by name in the registry, resolves path arguments via the path registry, and calls the implementation. No primitive code changes — only the invocation mechanism changes from Python API to command token.

11.2 Stage 2 → Stage 3

The scope resolver from Stage 2 gains mount-awareness in Stage 3. When walking the scope chain, if a mount point is encountered, the resolver follows the mount to the source KB (respecting mode restrictions). The query interface does not change — `scoped_query` still returns facts, but now it can find facts through mounts as well as through the parent chain.

The command executor from Stage 2 gains scratchpad integration in Stage 3. Internal computation results are automatically written to the scratchpad ring buffer. The inference loop reads the scratchpad to see what has been computed in the current turn.

11.3 Stage 3 → Stage 4

The inference loop from Stage 3 gains real tool execution in Stage 4. In Stage 3, the `formalize` step is a hook that accepts a callable — test code provides the formalized artifact directly. In Stage 4, the `formalize` step can generate command tokens that invoke operational primitives — fetch data from a URL, read a file, execute a script. The loop structure does not change. The tool vocabulary expands.

11.4 Stage 4 → Stage 5

The local environment from Stage 4 is joined by Docker, SSH, and VM environments in Stage 5. The executor does not change — it calls the same 10-method interface regardless of environment type. The environment selection is a configuration fact in the deployment KB. Switching from local to Docker is a fact change, not a code change.

12. Test Strategy

12.1 Test Layers

Unit tests verify individual functions against their IOSE declarations. Input the declared inputs, check the declared outputs, verify the declared side effects, confirm the declared properties. Every builtin has at least 3 unit tests (normal case, edge case, property verification).

Integration tests verify multi-function chains. A scoped query that walks the parent chain exercises the fact store, the scope resolver, and the KB store together. An inference loop iteration exercises the notebook, the loop, the command executor, the primitive registry, and the KB.

Lifecycle tests verify end-to-end scenarios. The toy lifecycle in Stage 1 (`init` → `train` → `checkpoint` → `evaluate` → `report`). The full lifecycle in Stage 5 (`source` → `corpus` → `tokenize` → `train` → `fine-tune` → `align` → `evaluate` → `deploy` → `monitor` → `update` → `retire`). Each lifecycle test verifies that the complete chain of KB operations produces the expected KB tree.

Invariant tests verify the cross-stage invariants from Section 5. Run at every stage. Confirm IOSE declarations exist for all registered builtins. Confirm no floats in computation paths. Confirm all data primitives respect their capacity bounds. Confirm idempotent operations satisfy $f(f(x)) = f(x)$.

12.2 Test Count Targets

Stage	Unit	Integration	Lifecycle	Invariant	Total	Cumulative
1	120	15	5	10	150	150
2	160	25	5	10	200	350
3	200	30	10	10	250	600
4	230	40	20	10	300	900
5	260	50	30	10	350	1,250

13. The Zig Port

The Zig 0.15.1 port happens after Stage 5 validates the complete system in Python. The port strategy:

Struct mapping. Every Python dataclass maps to a Zig struct. `VDRFraction` $\rightarrow$ `const VDRFraction = struct { v: i128, d: i128, r: Remainder }`. `KnowledgeBase` $\rightarrow$ `const KnowledgeBase = struct { ... }` with all 25 fields. The field types map directly: `Dict[str, Counter]` $\rightarrow$ `std.StringHashMap(Counter)`, `List[int]` $\rightarrow$ `std.ArrayList(i32)`, `Optional[int]` $\rightarrow$ `?i32`.

Enum mapping. Every Python Enum maps to a Zig enum. `Visibility` $\rightarrow$ `const Visibility = enum { public, internal, owner_only }`.

Function mapping. Every Python function maps to a Zig function with the same IOSE declaration. `def vdr_add(a: VDRFraction, b: VDRFraction) -> VDRFraction` $\rightarrow$ `fn vdr_add(a: VDRFraction, b: VDRFraction) VDRFraction`. The Result type maps to Zig's error unions: `Result[VDRFraction]` $\rightarrow$ `fn vdr_div(a: VDRFraction, b: VDRFraction) !VDRFraction`.

Arbitrary precision. Python's arbitrary-precision integers map to either Zig's `i128` (sufficient for most operations) with overflow to a `BigInt` library for operations that exceed 128 bits (Hilbert matrix determinants, Q335 numerators). The overflow boundary is detected and handled explicitly — not silently truncated.

Test mapping. Every Python test maps to a Zig test. `test "vdr_add commutative"` verifies the same property with the same inputs and expected outputs. The IOSE declarations are the test specifications — they are language-independent.

The port is mechanical because the IOSE declarations define the interfaces. The Zig implementation satisfies the same contracts. The same tests verify the same behavior. The Zig version runs faster, uses less memory, and has no garbage collection pauses — but it computes the same exact fractions and produces the same results.

14. Falsification Criteria

F1. If any stage produces a system that cannot execute its declared lifecycle capability end-to-end — the toy lifecycle fails in Stage 1, the full lifecycle fails in Stage 5 — the stage is incomplete.

F2. If any function is implemented without an IOSE declaration, the IOSE discipline has been violated. Every function must have its declaration before its implementation.

F3. If any cross-stage invariant fails at any stage — a float appears in the computation path, a data primitive exceeds its capacity, an idempotent operation is not idempotent — the invariant enforcement has a gap.

F4. If the test count at any stage falls below the target, the test coverage is insufficient for the claims made at that stage.

F5. If the Zig port of any function produces a different result from the Python prototype for the same inputs, the port has a correctness bug and the IOSE declaration did not adequately specify the behavior.

F6. If two stages' tests are incompatible — a Stage 2 change breaks a Stage 1 test — the incremental build discipline has been violated. Tests are cumulative and must never regress.

F7. If the supplementary function IOSE specification and the actual implementation disagree on inputs, outputs, or side effects for any function, the specification is the authority and the implementation has a bug.

15. Conclusion

Ten papers specified what the VDR-LLM-Prolog system does. This paper specifies how to build it.

Five stages. Stage 1 creates a toy with exact arithmetic, knowledge bases, Prolog rules, and a lifecycle loop. Stage 2 upgrades it with command tokens, paths, scoping, and constraints. Stage 3 adds sessions, inference, transcendentals, and domain mathematics. Stage 4 integrates operational environments, grants, external data access, all inference modes, and the lifecycle pipeline. Stage 5 completes the system with Docker, SSH, feed-back, deployment, monitoring, and retirement.

65 modules. 448 builtins. Approximately 15,500 new lines of Python, plus 5,000 lines of existing tested VDR-4 code. Every function has an IOSE declaration. Every stage produces a testable, runnable system. Every cross-stage invariant is verified at every stage.

The build is comprehensive (OSO C17) — the whole system is specified before any part is built. The build is incremental (OSO C3) — each stage removes a class of limitation. The build is disciplined (OSO C33) — one canonical method per task, one registration pattern per builtin, one test pattern per function.

The complete function-level IOSE specification is in the supplementary materials. The data structures, enums, and class definitions are in the supplementary materials. This paper is the map. The supplementary materials are the territory.

Build it in stages. Test it at every stage. Port it to Zig when the Python prototype validates every decision. The specification is complete. The plan is concrete. The system is buildable.

END HOWL-VDR-11-2026

Registry: [[HOWL-VDR-11-2026](#)]

Status: Specification complete

Domain: Applied Philosophy / Systems Engineering / Implementation Planning

Central Result: Five-stage build plan from toy to production. 65 modules, 448 builtins, ~20,500 total lines. Every function IOSE-declared. Python 3.8 prototype leveraging existing VDR-4 codebase, with mechanical port to Zig 0.15.1. Supplementary materials contain complete function-level specifications.

Foundation: VDR-1 through VDR-10

Key Principles: (1) Comprehensive specification before incremental build. (2) IOSE declaration before implementation. (3) Each stage is a complete, testable system. (4) Tests are cumulative and never regress. (5) The Python prototype validates; the Zig port performs.

Falsification: Seven criteria covering lifecycle completeness, IOSE discipline, invariant enforcement, test coverage, port correctness, regression prevention, and specification authority.

Supplementary Materials:

- `supplementary/function_iose_5_stages_spec.md`
- `supplementary/iose_spec_depth.md`

VDR-11 Extended Appendix Tables

Complete Reference Material for the Five-Stage Implementation Blueprint

Appendix A: Module Dependency Matrix

A.1 Import Dependencies by Module

Module	Imports From	Imported By	Stage
core/vdr.py	(stdlib only)	primitives/arithmetic, primitives/active arithmetic, primitives/structure ops, primitives/linalg builtins, primitives/statistics	Exists
core/active mul.py	core/vdr	primitives/active arithmetic	Exists
core/fn.py	core/vdr	primitives/functional, primitives/discrete calculus	Exists
core/linalg.py	core/vdr	primitives/linalg builtins, primitives/markov, primitives/graph math	Exists
core/types.py	core/vdr	All primitives (dispatch), command/executor	1
core/errors.py	(stdlib only)	All modules (Result type)	1
kb/knowledge base.py	core/errors	All kb/, all data primitives/, session/, inference/, lifecycle/	1
kb/fact store.py	kb/knowledge base	kb/rule engine, kb/scope resolver, command/executor	1
kb/rule engine.py	kb/knowledge base, kb/fact store, core/vdr	kb/constraint engine, inference/loop, inference/modes	1
kb/working data.py	kb/knowledge base	kb/scope resolver, session/snapshot	1
kb/constraint engine.py	kb/knowledge base, kb/rule engine	command/executor, inference/loop, lifecycle/training	2
kb/scope resolver.py	kb/knowledge base, kb/fact store, kb/working data, path/mount	command/executor, inference/loop	2
path/registry.py	core/errors	path/resolver, command/executor, session/snapshot	2
path/resolver.py	path/registry	command/executor, path/mount	2

Module	Imports From	Imported By	Stage
path/mount.py	path/registry, kb/knowledge base	kb/scope resolver	3
command/token types.py	(enums only)	command/parser, command/executor	2
command/parser.py	command/token types	command/executor	2
command/executor.py	command/parser, iose/registry, path/registry, kb/scope resolver, primitives/*	inference/loop	2
command/scratchpad.py	primitives/ring buffer	command/executor, inference/loop	2
session/snapshot.py	kb/knowledge base, data primitives/*	session/clone, session/lifecycle	3
session/clone.py	session/snapshot	session/lifecycle	3
session/lifecycle.py	session/snapshot, session/clone, kb/knowledge base	inference/loop	3
inference/notebook.py	kb/knowledge base, data primitives/*	inference/loop	3
inference/loop.py	inference/notebook, command/executor, kb/constraint engine	inference/modes	3
inference/confidence.py	core/vdr	inference/loop, inference/provenance	3
inference/provenance.py	kb/knowledge base, inference/confidence	inference/loop	3
inference/modes.py	kb/rule engine, inference/confidence	inference/loop	4
env/base.py	core/errors	env/local, env/docker, env/ssh, env/vm	4
env/local.py	env/base	ops/*, lifecycle/training	4
env/docker.py	env/base	ops/, lifecycle/	5
env/ssh.py	env/base	ops/, lifecycle/	5
env/vm.py	env/base	lifecycle/*	5
ops/grants.py	kb/knowledge base	ops/filesystem, ops/execution, ops/network, ops/process, ops/compilation, ops/linting	4
ops/filesystem.py	env/base, ops/grants	lifecycle/*, inference/modes	4
ops/execution.py	env/base, ops/grants	lifecycle/training, inference/modes	4

Module	Imports From	Imported By	Stage
ops/network.py	env/base, ops/grants	inference/modes, lifecycle/data pipeline	4
ops/process.py	env/base, ops/grants	lifecycle/training	4
ops/compilation.py	env/base, ops/grants	lifecycle/training	5
ops/linting.py	env/base, ops/grants	lifecycle/evaluation	5
lifecycle/data pipeline.py	kb/knowledge base, ops/network, ops/filesystem	lifecycle/training	4
lifecycle/training.py	kb/knowledge base, core/vdr, core/linalg, ops/execution	lifecycle/evaluation	4
lifecycle/evaluation.py	kb/knowledge base, ops/execution	lifecycle/deployment	4
lifecycle/feedback.py	kb/knowledge base, lifecycle/training	lifecycle/deployment	5
lifecycle/deployment.py	kb/knowledge base, env/base, lifecycle/evaluation	lifecycle/monitoring	5
lifecycle/monitoring.py	kb/knowledge base, ops/network	lifecycle/deployment	5
iose/registry.py	(stdlib only)	All primitives/ (registration), command/executor (lookup)	1
iose/validator.py	iose/registry	command/executor	2
iose/principles.py	kb/knowledge base, kb/fact store, core/vdr	(loaded at startup)	1

A.2 Circular Dependency Check

Check	Result	Notes
core/ → kb/	No	core has zero upward imports
kb/ → primitives/	No	kb does not import primitives
primitives/ → command/	No	primitives do not import command
command/ → session/	No	command does not import session
inference/ → ops/	No (Stage 3)	Stage 3 inference uses only kb + primitives + command

Check	Result	Notes
inference/ → ops/	Yes (Stage 4)	Stage 4 inference.modes imports ops for external tools — acceptable, both at same stage level
ops/ → lifecycle/	No	ops does not import lifecycle
lifecycle/ → lifecycle/	No	lifecycle modules have linear dependency
Verdict	No circular dependencies	All imports flow downward or laterally within same stage

Appendix B: Existing VDR-4 Codebase Inventory

B.1 Module-by-Module Wrapping Plan

Existing Module	Lines (est.)	Wrapped By	Wrapping Stage	Functions Exposed
vdr.py	~800	primitives/arithmetic.py		VDR. add , sub , mul , truediv , neg , abs , pow , normalize, compare
active mul.py	~400	primitives/active arithmetic.py		active add, active mul, active div
fn.py	~500	primitives/functional.py, primitives/discrete calculus.py		FnRemainder, resolve, make newton, make series, discrete derivative, left riemann, trapezoidal
linalg.py	~600	primitives/linalg builtins.py		Vec ops (6), Mat ops (15), det, inv, solve, rank
export.py	~150	primitives/conversion.py		to decimal, to float (at conversion boundary)
exp.py	~200	primitives/functional.py		exact exp (Taylor series)

Existing Module	Lines (est.)	Wrapped By	Wrapping Stage	Functions Exposed
logarithm.py	~200	primitives/functional.py		exact log (series)
basis.py	~300	primitives/qbasis.py		Q335 constants, qbasis add, qbasis mul
softmax.py	~200	primitives/statistics.py		softmax, softmax surrogate
autodiff.py	~400	lifecycle/training.py		backward, compute gradients
nn.py	~300	lifecycle/training.py		Linear, ReLU, Sequential
losses.py	~150	lifecycle/training.py		mse loss, l1 loss
optim.py	~200	lifecycle/training.py		SGD, Momentum
rng.py	~100	primitives/integer ops.py		deterministic random
init.py	~150	lifecycle/training.py		xavier rational
sampling.py	~200	lifecycle/training.py		categorical, top k, nucleus
datasets.py	~200	lifecycle/data 4 pipeline.py		tokenize, batch
metrics.py	~100	lifecycle/evaluation.py		accuracy, denom tracking
checkpoint.py	~200	lifecycle/training.py		save, load (exact VDR fractions)
tensor.py	~200	primitives/linear builtins.py		batched operations
attention.py	~300	lifecycle/training.py		scores, masking, weighting
transformer.py	~400	lifecycle/training.py		embedding, blocks, LM head
trainer.py	~300	lifecycle/training.py		training loop
Total	~5,550			

B.2 Wrapping Pattern

Every existing function is wrapped, not rewritten. The wrapper:

1. Accepts VDRFraction dataclass instances (the IOSE interface)
2. Converts to existing VDR class instances (the internal representation)
3. Calls the existing function

4. Converts the result back to `VDRFraction` dataclass
5. Returns `Result[VDRFraction]` (wrapping any exceptions as `Err`)

The conversion between `VDRFraction` (dataclass) and `VDR` (existing class) is a thin adapter:

```
VDRFraction(v=3, d=7, r=0) ↔ VDR(3, 7, 0)
```

Fields are identical. The adapter copies values, does not transform them.

Appendix C: Data Structure Size Estimates

C.1 Memory Footprint per Instance

Structure	Python Size (bytes, est.)	Zig Size (bytes, est.)	Notes
<code>VDRFraction</code> (closed, small ints)	120	40	Python over- head vs Zig struct
<code>VDRFraction</code> (closed, big ints ~100 digits)	300	160	<code>BigInt</code> storage
<code>VDRFraction</code> (active, 3 children)	600	200	Recursive struc- ture
<code>Fact</code> (3 args, short predicate)	400	80	Python string + list over- head
<code>Rule</code> (head + 2 body facts)	1,200	240	3 Facts
<code>Constraint</code>	500	120	Strings + enum
<code>Connection</code>	400	100	Strings + enum
<code>Counter</code>	200	16	+ ints 4 ints

Structure	Python Size (bytes, est.)	Zig Size (bytes, est.)	Notes
LockState	300	32	bool + optional string + optional int
BoundedQueue 400 (capacity 50, empty)		64	List over-head
BoundedQueue 2,400 (capacity 50, full)		2,064	50 × ~40 bytes per item
BoundedStack 300 (capacity 30, empty)		48	List over-head
RingBuffer 500 (capacity 100, empty)		80	List + 2 ints
LRUCache 600 (capacity 50, empty)		128	Dict + list over-head
Bitset (width 100)	1,200	16	Python bool list vs Zig packed bits
KnowledgeBase 4,000 (empty)		800	All field initialization
KnowledgeBase 30,000 (moderate: 50 facts, 10 rules, 5 primitives)		5,000	Content dominates
SessionSnapshot 50,000 (10 KBs, moderate state)		10,000	Deep copy of live state

Structure	Python Size (bytes, est.)	Zig Size (bytes, est.)	Notes
Inference Notebook (active, mid-investigation)	80,000	15,000	KB + counters + queues + LRUs + evidence

C.2 Scaling Estimates

Scenario	KBs	Facts (total)	Python Memory	Zig Memory
Toy lifecycle (Stage 1)	5	50	~200 KB	~30 KB
Upgraded toy (Stage 2)	15	200	~1 MB	~150 KB
Active inference (Stage 3)	30	500	~3 MB	~500 KB
Full lifecycle (Stage 4)	100	2,000	~15 MB	~3 MB
Production with history (Stage 5)	500	10,000	~80 MB	~15 MB
Large deployment (post Stage 5)	2,000	50,000	~400 MB	~80 MB

Python’s overhead is approximately $5\text{-}6 \times$ Zig for the same logical content. The Zig port provides significant memory efficiency for production workloads.

Appendix D: Test Plan Detail

D.1 Stage 1 Test Breakdown

Test File	Module Tested	Test Count	Test Types
test vdr wrapping.py	primitives/arithmetic	10	Unit: add, sub, mul, div, neg, abs, pow, reciprocal + 2 property (commutative, associative)
test comparison.py	primitives/comparison	10	Unit: compare, equal, lt, le, min, max, sign, is zero, is positive, is negative

Test File	Module Tested	Test Count	Test Types
test rounding.py	primitives/rounding	8	Unit: floor, ceil, round, truncate, numerator, denominator, simplify + idempotency
test aggregates.py	primitives/list aggregates	8	Unit: sum, product, mean, dot product, sum sq, weighted, harmonic, alternating
test fact store.py	kb/fact store	10	Unit: assert, retract, query, exists + idempotency (assert existing, retract missing)
test rule engine.py	kb/rule engine	15	Unit: unify atoms, unify vars, unify fractions, unify lists, unify facts, query single, query multiple, backtrack, depth limit, findall
test working data.py	kb/working data	5	Unit: set, get, get local, delete, list visible
test kb lifecycle.py	kb/knowledge base	5	Integration: create, get, delete, parent-child, orphan detection
test counter.py	data primitives/counter	6	Unit: create, inc, dec, add, reset, set + clamp at bounds
test lock.py	data primitives/lock	5	Unit: create, acquire, release, check, force release + acquire-on-held

Test File	Module Tested	Test Count	Test Types
test queue.py	data primitives/queue	6	Unit: create, push, pop, peek, clear, to list + push-when-full
test stack.py	data primitives/stack	5	Unit: create, push, pop, peek, clear + push-when-full
test text.py	primitives/text	17	Unit: one per builtin (reverse, length, concat, split, slice, char at, to chars, chars to, contains, starts, ends, upper, lower, trim, replace, join, pad left)
test collections.py	primitives/collections	15	Unit: sample of 15 critical ops (sort, filter, map, reduce, zip, unique, head, tail, nth, take, drop, partition, group by, frequencies, interleave)
test sets.py	primitives/sets	5	Unit: from list, union, intersection, difference, is subset
test mappings.py	primitives/mappings	5	Unit: from pairs, get, set, merge, invert
test conversion.py	primitives/conversion	5	Unit: to string, parse json, format json, to fraction, from decimal string
test logic.py	primitives/logic	3	Unit: if then else, try catch, findall
test integer ops.py	primitives/integer ops	5	Unit: add, mul, mod, range, bit and

Test File	Module Tested	Test Count	Test Types
test iose registry.py	iose/registry	5	Unit: register, get by id, get by name, all in category, count
test principles.py	iose/principles	5	Unit: load, get knowability, get priority, priority winner + check axioms present
test toy lifecycle.py	Integration	5	Lifecycle: create KB tree, init model, train step, checkpoint, evaluate
Total		161	

D.2 Stage 2 Test Breakdown

Test File	Module Tested	Test Count	Key Tests
test constraint engine.py	kb/constraint engine	10	check single, check all, enforce warn, enforce block, add, remove, enable, suspend, idempotent enable, idempotent suspend
test scope resolver.py	kb/scope resolver	15	build chain, scoped query local, scoped query inherited, scoped query shadowed, scoped query all, resolve binding scoped, secondary scope, out of scope invisible

Test File	Module Tested	Test Count	Key Tests
test path registry.py	path/registry	10	register, resolve, from id, exists, sequential ids, never reuse retired, stable across calls
test path resolver.py	path/resolver	10	parent, children, ancestors, depth, common ancestor, root has no parent, depth limit
test command parser.py	command/parser	15	parse pure fn, parse kb assert, parse kb query, parse path ref, parse literal int, parse literal fraction, parse mixed stream, parse malformed graceful, parse empty
test command executor.py	command/executor	15	execute pure, execute kb assert, execute kb query, path resolution, type validation, chain execution, scratchpad recording, error on unknown primitive, error on unknown path
test scratchpad.py	command/scratchpad	5	write, read recent, clear, capacity limit, automatic overwrite

Test File	Module Tested	Test Count	Key Tests
test active arithmetic.py	primitives/active arithmetic	10	same d add, diff d add, active mul, div by closed, div by active, remainder preservation, v1 compromise documented
test structure ops.py	primitives/structure ops	8	lift atomic, lift composite, lift child, lift composition, rebase closed, rebase active, mismatch witness, projection closed
test number theory.py	primitives/number theory	13	gcd, lcm, mod, div exact, mod pow, mod inv, ext gcd, is prime, factorial, binomial, fibonacci, totient, crt
test linalg builtins.py	primitives/linalg builtins	15	vec add, vec dot, vec norm sq, mat mul, mat det, mat inv, mat solve, mat rank, mat pow, gram schmidt, hilbert 3x3 exact, hilbert 4x4 exact, inv inv identity
test statistics.py	primitives/statistics	10	mean, variance, median, softmax sum one, surrogate sum one, normalize sum one, prob bayes, prob cdf, expected value

Test File	Module Tested	Test Count	Key Tests
test time ops.py	primitives/time ops	5	from ymd, to ymd roundtrip, diff days, day of week, leap year
test identity.py	primitives/identity	5	hash deterministic, base64 roundtrip, hex roundtrip, crc32, uuid
test graphs.py	primitives/graphs	10	deterministic from edges, bfs, dfs, shortest path, components, connected, topo sort, cycle detect, mst, pagerank
test lru.py	data primitives/lru	8	create, push, get, peek, contains, eviction on full, access updates
test ring buffer.py	data primitives/ring buffer	6	order, clear create, write, read all, read last, overwrite oldest, clear
test bitset.py	data primitives/bitset	8	create, set, clear bit, test, count, all set, any set, to list
test iose validator.py	iose/validator	8	type compatible ok, type incompatible caught, se preview, contract satisfied, contract violated undeclared, contract violated missing
test command lifecycle.py	Integration	5	Full cycle via command tokens: assert, query, compute, store, query

Test File	Module Tested	Test Count	Key Tests
test scope lifecycle.py	Integration	5	Create tree, switch topic, verify visibility, verify invisibility, cross scope query
Total	191		

D.3 Stage 3 Test Breakdown

Test File	Module Tested	Test Count	Key Tests
test mount.py	path/mount	10	create, remove, cycle detect, resolve through read only, block write on read only, snapshot mount frozen, mirror sees changes
test qbasis.py	primitives/qbasis	10	add same exp, sub, mul with error bound, scalar mul, to fraction lossless, get constant pi, get constant e, precision bits, pi plus e integer add
test functional.py	primitives/functional	12	sqrt2 depth7, exp depth12, log positive, log fails negative, sin depth10, cos depth10, resolve newton, resolve series, make newton, make series, sqrt2 convergence rate

Test File	Module Tested	Test Count	Key Tests
test discrete calculus.py	primitives/discrete calculus	10	derivative x squared, nth derivative, left riemann x squared, trapezoidal x squared, finite diff cubic, finite diff zero beyond degree, richardson
test denom mgmt.py	primitives/denom mgmt	8	denom bits, denom digits, reproject qbasis bounded, budget check under, budget check over, precision state closed, precision state active
test polynomial.py	primitives/polynomial	10	eval horner, add, mul, div, gcd, derivative, integral, lagrange interpolation, lagrange exact recovery
test finite field.py	primitives/finite field	6	gf add, gf mul, gf inv, gf pow, all inverses in gf7, inv fails at zero
test markov.py	primitives/markov	6	steady state sum one, steady state exact, step, n steps, n steps
test graph math.py	primitives/graph math	4	equals mat pow adjacency power walk count, pagerank sum one, pagerank exact via cramer

Test File	Module Tested	Test Count	Key Tests
test snapshot.py	session/snapshot	10	capture live state, create snapshot, restore exact, restore idempotent, snapshot independence (modify source, snapshot unchanged), snapshot does not capture persistent
test clone.py	session/clone	10	clone from snapshot, clone live state independent, clone persistent shared, kill clone, kill preserves persistent, clone counter isolated, clone queue isolated
test session lifecycle.py	session/lifecycle	8	reset clears all live, reset preserves persistent, list snapshots, diff snapshots, info, reset idempotent
test notebook.py	inference/notebook	8	create with schema, create from template sre, create from template research, standard primitives present, goal set, status active

Test File	Module Tested	Test Count	Key Tests
test inference loop.py	inference/loop	15	assess identifies gap, formalize produces artifact, execute runs artifact, store persists result, budget exhaustion halts, stall detection backtracks, goal satisfaction concludes, full loop 3 steps, evidence counter incremented, steps since evidence reset
test confidence.py	inference/confidence	8	deductive min, inductive coverage times mean, abductive explained fraction, analogical product, chain propagation, exact fractions throughout
test provenance.py	inference/provenance	8	record evidence, record conclusion, trace derivation, challenge invalidates, challenge partial impact, derivation chain complete

Test File	Module Tested	Test Count	Key Tests
test session inference integration.py	Integration	5	snapshot before inference, clone runs inference, clone results persist, kill clone preserves conclusion
test math integration.py	Integration	5	qbasis in inference, functional in inference, discrete calc in inference
Total	163		

D.4 Stage 4-5 Test Summary

Stage	Test Category	Test Count	Key Focus Areas
4	Grant system	20	Verify, use, expire, exhaust, inherit through hierarchy, deny on missing
4	Filesystem ops	30	Read, write, exists, list, create dir, delete, glob, diff + grant enforcement
4	Script execution	20	Python, shell, pytest + async + error handling + grant enforcement
4	Network ops	15	fetch, post, download + timeout + grant + JSON parsing chain
4	Process mgmt	15	start, poll, wait, kill, stdout, stderr + background completion
4	Inference modes	40	Deductive: premises→conclusion. Inductive: evidence→ranking. Abductive: symptoms→causes. Analogical: mapping→transfer. Mode composition chains.

Stage	Test Category	Test Count	Key Focus Areas
4	Lifecycle pipeline	60	source→corpus→tokenize→train→checkpoint→evaluate. Each phase produces correct KB tree.
4	Local environment	20	Cross-phase queries work. All 10 interface methods.
4	Integration	40	File persistence. Process lifecycle. Full inference with external data. Lifecycle with evaluation gates.
4	IOSE contracts	40	Grant enforcement end-to-end. All Stage 4 builtins verified against declarations
Stage 4 subtotal		300	
5	Docker environment	40	Container lifecycle. Isolation verification. Mount points. Resource limits. Startup script.
5	SSH environment	30	Connection. Key auth. Remote exec. File transfer. Timeout.
5	VM environment	20	Create. Provision. Snapshot. Destroy.
5	Compilation	15	Python syntax check. Zig compile. C compile. Rust compile. Error reporting.
5	Linting	20	Python lint. JSON validate. Complexity analysis. Import analysis.
5	Feedback	30	Pairwise judgment. Agreement computation exact. Reward model training. DPO.
5	Deployment	30	Create config. Canary. Promotion criteria exact. Rollback.
5	Monitoring	30	Watch creation. Metric recording. Drift detection. Alert triggering.

Stage	Test Category	Test Count	Key Focus Areas
5	Canary + rollback	25	Canary traffic split. Promotion. Auto-rollback on regression. Rollback idempotent.
5	Retirement	15	Archive. Frozen but queryable. Successor link. No active deployments.
5	Full lifecycle	40	Raw data → trained → deployed → monitored → updated → retired. Single KB tree.
5	End-to-end	25	Complete scenario: source bad data, train, detect quality issue, retrain, deploy, canary, promote.
5	Zig port prep	30	Type mapping verification. Dataclass↔struct equivalence. Result↔error union.
Stage 5 subtotal		350	

Appendix E: Builtin Registration Tables

E.1 Stage 1 Builtins (151 builtins)

ID Range	Category	Count	Registration Pattern
1-8	VDR closed arithmetic	8	Wrap vdr.py class methods
9-18	Comparison	10	Pure, VDRFraction in, simple out
19-25	Rounding/extraction	7	Pure, VDRFraction in, int or bool out
26-33	List aggregates	8	Pure, List[VDRFraction] in, VDRFraction out
34-50	Text	17	Pure, string in, string/bool/int/list out
51-86	Collections	36	Pure, list in, list/item/bool/int out
87-100	Sets	14	Pure, set in, set/bool/int out
101-115	Mappings	15	Pure, dict in, dict/value/list out
116-129	Conversion	14	Pure (some partial), mixed in/out
130-140	Logic	11	Pure, mixed in/out

ID Range	Category	Count	Registration Pattern
141-161	Integer + bit ops	21	Pure, int in, int/bool/list out
Total		161	

Note: Stage 1 target was ~150. Actual enumeration produces 161 due to complete category coverage.

E.2 Stage 2 Additional Builtins (+139, cumulative 300)

ID Range	Category	Count	Registration Pattern
162-166	Active arithmetic	5	Wrap active mul.py
167-169	Structure ops	3	Lift, rebase, projection
170-182	Number theory	13	Pure, int in, int out
183-206	Linear algebra	24	Wrap linalg.py
207-222	Statistics + probability	16	Pure, list/fraction in, fraction/list out
223-232	Time	10	Pure, int in, int/str out
233-240	Identity	8	Pure, deterministic
241-253	Graphs	13	Pure, graph in, list/bool/int out
254-268	KB operations	15	KB-internal side effects
269-289	Data primitives (LRU+ring+bitset)	21	KB-internal side effects
290-297	Path operations	8	Pure
298-300	Scratchpad + IOSE validator	3	Internal
Total		139	

E.3 Stage 3 Additional Builtins (+100, cumulative 400)

ID Range	Category	Count
301-307	Q-basis	7
308-315	Functional remainder	8
316-321	Discrete calculus	6
322-326	Denominator management	5
327-334	Polynomial	8
335-338	Finite field	4
339-341	Markov	3
342-343	Graph math	2
344-360	Mount operations	9+8 = path/mount + session ops
Total		100

E.4 Stage 4 Additional Builtins (+37, cumulative 437)

ID Range	Category	Count
361-375	Filesystem	15
376-380	Execution	5
381-385	Network	5
386-392	Process	7
393-397	Grant operations	5
Total		37

E.5 Stage 5 Additional Builtins (+11, cumulative 448)

ID Range	Category	Count
398-401	Compilation	4
402-409	Linting	8
Total		12

Note: Stage 5 also adds lifecycle functions (feedback, deployment, monitoring) but these are system operations rather than registered builtins — they compose builtins rather than being builtins themselves.

Appendix F: Python 3.8 Compatibility Checklist

F.1 Language Features to Avoid

Feature	Introduced In	Avoidance Strategy
Walrus operator <code>:</code>	3.8 (limited)	Avoid in comprehensions; use explicit assignment
f-string = debug	3.8+ only for = suffix	Use <code>"name: %s" % name</code> or <code>"name: {}".format(name)</code>
match statement	3.10	Use if/elif chains or dict dispatch
type alias syntax	3.12	Use typing module: <code>List</code> , <code>Dict</code> , <code>Optional</code>
for union types	3.10	Use <code>Union[A, B]</code> from typing
@dataclass (slots=True)	3.10	Omit slots parameter
@dataclass (kwonly=True)	3.10	Omit; use default values
Positional-only parameters /	3.8	Available but avoid for portability

Feature	Introduced In	Avoidance Strategy
dict dict merge	3.9	Use <code>{**a, **b}</code> or dict merge builtin
list[int]	3.9	Use <code>List[int]</code> from typing
lowercase generics		
str.removeprefix	3.9	Use <code>s[len(prefix):]</code> after startswith check

F.2 Required Imports Pattern

```

from __future__ import annotations # Enable postponed evaluation for forward
from dataclasses import dataclass, field
from typing import List, Optional, Dict, Tuple, Any, Callable, Union
from enum import Enum
from fractions import Fraction # For VDR scalar projection comparison
import copy # For session snapshot deep copy
import json # For parse_json/format_json
import os # For filesystem ops
import subprocess # For execution ops (Stage 4+)

```

Appendix G: Zig Port Type Mapping Reference

G.1 Complete Type Mapping

Python Type	Zig Type	Notes
int (small)	i32 or i64	Counter values, IDs, indices
int (arbitrary precision)	i128 with BigInt overflow	VDR numerator/denominator, Q335 numerators
bool	bool	Direct
str	[]const u8 or std.ArrayList(u8)	Immutable vs mutable

Python Type	Zig Type	Notes
float	Never used	System is exact-only
None	void	Return type
Optional[T]	?T	Nullable
List[T]	std.ArrayList(T)	Dynamic array
Dict[str, T]	std.StringHashMap(T)	String-keyed map
Dict[int, T]	std.AutoHashMap(i32, T)	Integer-keyed map
Tuple[A, B]	struct { a: A, b: B }	Named fields
Union[A, B]	union(enum) { a: A, b: B }	Tagged union
Callable[[A], B]	fn (A) B or *const fn(A) B	Function pointer
Enum	const MyEnum = enum { ... }	Direct
@dataclass	const MyStruct = struct { ... }	Direct
Result[T] (custom)	MyError!T (error union)	Zig-native error handling
try/except	catch/errdefer	Different syntax, same semantics
copy.deepcopy(x)	Explicit alloc + field copy	Manual but straightforward
list comprehension	for loop with append	Mechanical translation
dict comprehension	for loop with put	Mechanical translation
@property	Zig method on struct	pub fn name(self: *const Self) T

G.2 Performance-Critical Mapping Decisions

Decision	Python Approach	Zig Approach	Rationale
VDR numerator/denominator	Python arbitrary-precision int	i128 with overflow to BigInt	99% of values fit in i128; BigInt only for extreme cases (Hilbert, Q335 products)

Decision	Python Approach	Zig Approach	Rationale
KB fact storage	List[Fact] with linear scan	ArrayList(Fact) with hash index on predicate	Linear scan is $O(n)$ per query; hash index is $O(1)$ average
Path registry Scope chain	Dict[str, int] List[int]	StringHashMap(i32) [16]i32 stack-allocated array	Direct mapping Max depth 16 (from VDR-8 spec); avoids allocation
Bitset	List[bool]	std.StaticBitSet(width) or std.DynamicBitSet	Zig has native packed bit operations
Ring buffer	List with modular indexing	Fixed-size array with write pos	Stack-allocated for small capacity
Deep copy for snapshots	copy.deepcopy	Arena allocator + structured copy	Zig arenas make bulk allocation/deallocation efficient
String interning for predicates	(no interning)	StringPool with integer handles	Reduces memory and enables integer comparison

Appendix H: Risk Registry

H.1 Technical Risks

Risk	Impact	Likelihood	Mitigation	Stage Affected
Rule engine too slow for large KBs	Query latency increases with fact count	Medium	Add predicate index (hash map) at Stage 2. Linear scan acceptable for Stage 1 toy sizes.	2+

Risk	Impact	Likelihood	Mitigation	Stage Affected
Denominator growth exceeds memory during training	OOM on long training runs	Medium	Denominator management builtins (Stage 3). Budget constraints trigger reprojection. Tested in VDR-1 gyms.	3+
Unification on active VDR objects is complex	Edge cases in matching active remainders	Low	Active objects are projected to closed rationals for comparison (existing VDR behavior).	1
Command token parsing ambiguity	CMD: prefix appears in user text	Low	Strict prefix syntax: CMD: must be at line start, followed by known primitive name. Unknown primitives are treated as text.	2
Session snapshot too large	Snapshot of large live state is slow	Low	Data primitives are bounded (max capacity). Snapshot size is bounded by sum of capacities $\times$ entry size. Estimates in Appendix C show <500KB even for large sessions.	3

Risk	Impact	Likelihood	Mitigation	Stage Affected
Python proto-type too slow for realistic training	Training loops with exact fractions are orders of magnitude slower than float	High (expected)	Prototype validates correctness, not performance. Training on toy models (2-dim, 3-vocab). Production performance comes from Zig port.	All
Docker API changes between Python proto-type and Zig port	Docker client library differs	Low	Use subprocess-based Docker CLI wrapper, not library bindings. CLI is stable across languages.	5
Q335 numerator multiplication overflow	Product of two 102-digit integers is 204 digits	None (Python handles it)	Python arbitrary precision handles natively. Zig port needs BigInt for this specific operation.	3

H.2 Process Risks

Risk	Impact	Likelihood	Mitigation
Scope creep at any stage	Stage never completes	Medium	Each stage has a fixed module list and test count. Stage is done when all tests pass. No new modules added mid-stage.
Tests pass but IOSE not verified	IOSE becomes decorative	Medium	Invariant tests at every stage check that all registered builtins have IOSE declarations. IOSE validator (Stage 2) checks contracts.

Risk	Impact	Likelihood	Mitigation
Existing VDR-4 code has undocumented behavior	Wrapper produces unexpected results	Low	VDR-4 has 705 tests. Wrapping tests verify same inputs produce same outputs through the wrapper.
Zig port reveals Python-specific assumptions	Some Python idioms don't translate	Medium	Avoid Python-specific patterns from the start. No operator overloading in builtins (use named functions). No exceptions (use Result). No global state (pass KB store explicitly).

Appendix I: Glossary of Implementation Terms

Term	Definition	First Appears
Builtin	A registered primitive function with IOSE declaration. One of 448 operations the system provides.	VDR-6, formalized VDR-10
IOSE Declaration	Structured record of a function's Inputs, Outputs, Side Effects, and Properties.	VDR-10
KB Store	The global dictionary mapping integer IDs to KnowledgeBase instances. Single source of truth.	VDR-11 Stage 1
Result Type	An Ok/Err wrapper returned by functions that can fail. Replaces exceptions for expected failures.	VDR-11 Stage 1
Wrapping	Creating an IOSE-declared builtin function that delegates to an existing VDR-4 implementation.	VDR-11 Stage 1

Term	Definition	First Appears
Registration Pattern	The table-driven process of declaring and registering builtins with the global BuiltinRegistry.	VDR-11 §8
Scope Chain	Ordered list of KB IDs from active topic to root. Determines which facts are visible.	VDR-5, implemented Stage 2
Turn Counter	Global integer incremented per user interaction. Provides total ordering of events.	VDR-11 §10.3
Conversion Boundary	The declared point where external approximate data enters the exact VDR system, with logged error bound.	VDR-10, implemented Stage 1
Toy Lifecycle	The minimal init→train→checkpoint→evaluate→report cycle demonstrated in Stage 1.	VDR-11 §4.1
Disposable Clone	A session clone launched from a stable snapshot, expected to be killed and relaunched when drift is detected.	VDR-8, implemented Stage 3
Inference Notebook	A KB subtree with standard schema housing one orchestrated inference investigation.	VDR-9, implemented Stage 3

Appendix J: Cumulative System Statistics

J.1 Complete Paper Series

Paper	Registry	Central Result
VDR-1	[HOWL-VDR-1-2026]	Exact arithmetic in irreducible triple form
VDR-2	[HOWL-VDR-2-2026]	15 domains, 282 tests
VDR-3	[HOWL-VDR-3-2026]	23 domains, transcendental integration
VDR-4	[HOWL-VDR-4-2026]	24-module ML stack, working exact transformer

Paper	Registry	Central Result
VDR-5	[HOWL-VDR-5-2026]	Prolog KB architecture, constraints, scoped knowledge
VDR-6	[HOWL-VDR-6-2026]	255 primitives, command tokens, operational environments
VDR-7	[HOWL-VDR-7-2026]	12-phase lifecycle, training through retirement
VDR-8	[HOWL-VDR-8-2026]	Data primitives, dotted paths, session management
VDR-9	[HOWL-VDR-9-2026]	Orchestrated Inference
VDR-10	[HOWL-VDR-10-2026]	IOSE system model, operational principles, 448 builtins
VDR-11	[HOWL-VDR-11-2026]	Five-stage implementation blueprint

J.2 System Capability Progression

Paper	What It Added	Cumulative Capability
VDR-1–4	Exact arithmetic + ML stack	Can compute exactly
VDR-5	Knowledge bases, constraints, scoping	Can know and constrain
VDR-6	Primitives, commands, environments	Can do and execute
VDR-7	Lifecycle management	Can train, deploy, and retire
VDR-8	Runtime state, addressing, sessions	Can remember, address, and recover
VDR-9	Orchestrated Inference	Can investigate, reason, and conclude
VDR-10	IOSE model, engineering principles, full math	Can be specified for building
VDR-11	Five-stage build plan	Can be built

J.3 Final System Dimensions

Dimension	Count
Papers in series	11 (plus MATH-3, MATH-4)
Total builtins	448
Builtin categories	25
KB struct fields	25
Implementation modules	65
Existing code (VDR-4)	~5,500 lines
New code (Stages 1-5)	~15,500 lines

Dimension	Count
Total prototype code	~21,000 lines
Planned tests	1,250
Existing passing tests	705
OSO principles in root KB	~176 Prolog terms
Inference modes	4
Environment types	4 (local, Docker, SSH, VM)
Lifecycle phases	12
Number types	5 (VDR fraction, integer, decimal display, Q-basis, functional remainder)
IOSE-declared components	All 448 builtins + 11 system components

END VDR-11 EXTENDED APPENDIX TABLES

[[HOWL-VDR-11-2026](#)] Implementation Blueprint. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/11-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-3-2026](#)] VDR Gym Extension. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-3-2026>

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026>

[[HOWL-LLM-1-2026](#)] Integer LLM with Prolog Knowledge Base. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026>

[[HOWL-VDR-5-2026](#)] Exact Arithmetic Meets Logical Provenance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026>

[[HOWL-VDR-6-2026](#)] Computational Primitives and Operational Environments. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-6-2026>

[[HOWL-VDR-7-2026](#)] Complete Lifecycle Technical Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-7-2026>

[[HOWL-VDR-8-2026](#)] Computational State Primitives, Universal Data Pathing, and Session Management. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026>

[[HOWL-VDR-9-2026](#)] Orchestrated Inference. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-9-2026>

[[HOWL-VDR-10-2026](#)] Operational Foundations and Comprehensive Builtin Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-10-2026>